

Звіт

до лабораторної роботи №5
Об'єктно-орієнтоване програмування. Java
Успадкування Java

Студент: Ісаєв Захар Ігорович

Група: КН-924В

Викладач: Володимир Миколайович Савченко

Харків - 2026

Мета роботи

Сформуванати практичні навички побудови семантично коректних ієрархій класів у Java, реалізації багаторівневого успадкування без абстрактних класів та інтерфейсів, використання конструкторів, `super`, `@Override` і `final`, а також інтерпретації структури об'єкта в пам'яті за допомогою JOL.

Опис предметної галузі

У даній лабораторній роботі розглядається предметна область медичних установ і персоналу. Для моделювання було обрано ієрархію медичних працівників, оскільки між цими класами існує коректний зв'язок «є різновидом».

У попередній ЛР2 була реалізована система бронювання номерів у готелі з класами `Guest`, `Room`, `Hotel`, `Reservation` та `Main`. У тій моделі переважали зв'язки типу «має»: `Hotel` має кімнати та бронювання, а `Reservation` пов'язує гостя, кімнату і дату. Тому для ЛР2 більш доречними були асоціація, агрегація та композиція.

У ЛР5 побудовано нову ієрархію: `MedicalWorker` -> `Doctor` -> `Surgeon` -> `CardiacSurgeon`. Також додано відгалуження `Doctor` -> `Pediatrician`. Клас `MedicalWorker` описує загального медичного працівника, `Doctor` уточнює його як лікаря, `Surgeon` — як хірурга, а `CardiacSurgeon` — як кардіохірурга. `Pediatrician` є окремим різновидом лікаря, але не є хірургом.

Побудова ієрархії класів

Базовий клас `MedicalWorker`

Клас `MedicalWorker` є базовим у створеній ієрархії. У ньому зосереджено спільні для всіх медичних працівників поля `fullName`, `hospitalName` і `experienceYears`. Цей клас описує загального працівника медичної установи та містить метод `description()`, який формує текстове представлення об'єкта.

Клас `Doctor`

Клас `Doctor` успадковує `MedicalWorker` за допомогою `extends`. Він уточнює базовий клас і представляє лікаря як різновид медичного працівника. У класі додаються власні поля `specialization` і `licenseNumber`, які описують

спеціалізацію лікаря та номер його ліцензії. Для ініціалізації успадкованих полів у конструкторі використано `super(...)`. Метод `description()` перевизначено за допомогою `@Override`.

Клас Surgeon

Клас Surgeon є нащадком Doctor. Він описує хірурга як спеціалізований різновид лікаря. У цьому класі додаються поля `operationsCount` і `operatingRoom`, які зберігають кількість проведених операцій та операційну кімнату. У конструкторі використано `super(...)` для передачі даних у клас Doctor. Метод `description()` знову перевизначено, щоб додати до опису інформацію про хірургічну діяльність.

Клас CardiacSurgeon

Клас CardiacSurgeon успадковує Surgeon та є кінцевим класом основної ієрархії. Він описує кардіохірурга, тобто хірурга, який спеціалізується на операціях на серці. У класі додано власні поля `heartOperationsCount` і `usesHeartLungMachine`. Клас оголошено як `final`, тому подальше успадкування від нього заборонене. Це демонструє використання `final` у контексті ієрархії класів.

Клас Pediatrician

Клас Pediatrician є окремим відгалуженням від Doctor. Він описує педіатра як різновид лікаря, який працює з дітьми. У класі додаються поля `maxPatientAge` і `vaccinationCertified`, що зберігають максимальний вік пацієнтів та інформацію про сертифікацію з вакцинації. Клас не успадковує Surgeon, тому що педіатр не є різновидом хірурга. Це робить ієрархію семантично коректною.

У створеній ієрархії використано такі можливості Java:

Механізм	Приклад використання
extends	успадкування Doctor від MedicalWorker, Surgeon від Doctor, CardiacSurgeon від Surgeon, Pediatician від Doctor
super	виклик конструктора базового класу для ініціалізації успадкованих полів
@Override	перевизначення методу description() у класах-нащадках
final	final-клас CardiacSurgeon, від якого заборонено подальше успадкування

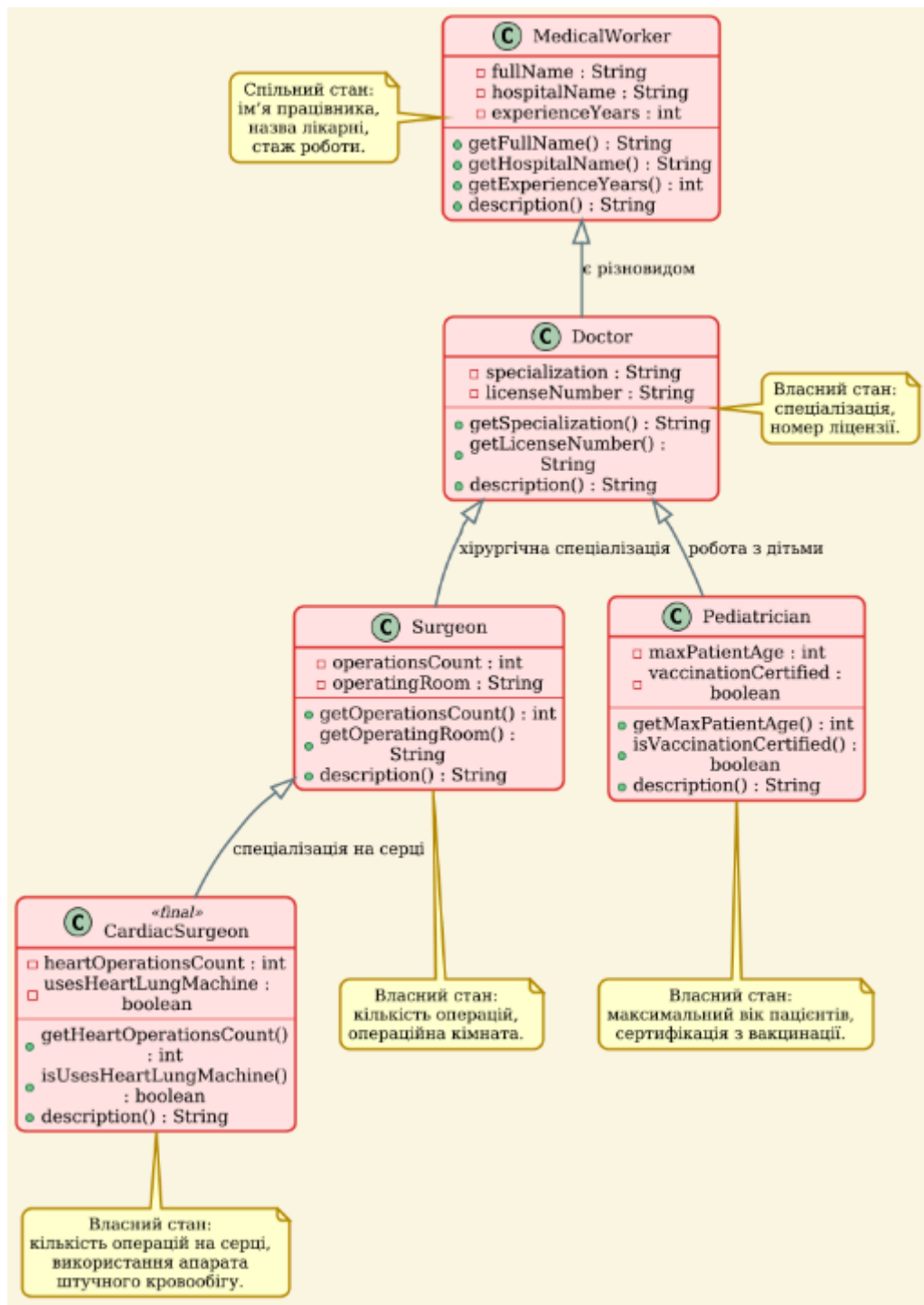


Рис. 1 - UML-діаграма класів

Робота програми

Під час виконання програми створюються об'єкти різних рівнів ієрархії медичних працівників: `MedicalWorker`, `Doctor`, `Surgeon`, `CardiacSurgeon` та `Pediatrician`. Після створення кожного об'єкта програма виводить його опис у консоль за допомогою методу `description()`. Це дозволяє побачити, що

кожен наступний клас успадковує поля батьківського класу та додає власний стан. Наприклад, Doctor додає спеціалізацію і номер ліцензії, Surgeon додає кількість операцій та операційну кімнату, а CardiacSurgeon додає інформацію про операції на серці. Таким чином, програма підтверджує коректну роботу побудованої ієрархії успадкування для предметної області медичного персоналу.

```

C:\Users\kurtz\jdk\openjdk-25.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=65020*" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath D:\IntelRobotif\oop-khpi
MedicalWorker: Danya Varenik, hospitalName=Kharkiv City Hospital, experienceYears=3
MedicalWorker: Kolya Kamazyska, hospitalName=Kharkiv City Hospital, experienceYears=8, specialization=Therapy, licenseNumber=DOC-1024
MedicalWorker: Beha Doto, hospitalName=Regional Clinical Hospital, experienceYears=12, specialization=Surgery, licenseNumber=SUR-2048, operationsCount=560, operatingRoom=Operating room 3
MedicalWorker: Chotyk Sergeivich, hospitalName=Cardiology Center, experienceYears=15, specialization=Cardiac surgery, licenseNumber=CARD-4096, operationsCount=820, operatingRoom=Operating room 1, heartOperationsCount=370, u
MedicalWorker: Sanya Vasyan, hospitalName=Children Hospital, experienceYears=10, specialization=Pediatrics, licenseNumber=PED-3904, maxPatientAge=18, vaccinationCertified=true
Process finished with exit code 0

```

Рис. 2- Робота програми

```

va.khpi.oop.lab05.CardiacSurgeon object internals:
OFF  SZ                TYPE DESCRIPTION                                 VALUE
  0   8                (object header: mark)                        0x0000000000000001 (non-biasable; age: 0)
  8   4                (object header: class)                      0x010438e8
 12   4                int MedicalWorker.experienceYears          15
 16   4  java.lang.String MedicalWorker.fullName                  (object)
 20   4  java.lang.String MedicalWorker.hospitalName              (object)
 24   4  java.lang.String Doctor.specialization                   (object)
 28   4  java.lang.String Doctor.licenseNumber                    (object)
 32   4  java.lang.String Surgeon.operatingRoom                   (object)
 36   4                int Surgeon.operationsCount                820
 40   4                int CardiacSurgeon.heartOperationsCount     370
 44   1                boolean CardiacSurgeon.usesHeartLungMachine true
 45   3                (object alignment gap)
Instance size: 48 bytes
Space losses: 0 bytes internal + 3 bytes external = 3 bytes total

```

Рис. 3 - JOL-аналіз

Висновок

У лабораторній роботі №5 було реалізовано ієрархію класів для предметної області медичних установ і персоналу. Було побудовано основний ланцюг успадкування MedicalWorker -> Doctor -> Surgeon -> CardiacSurgeon та додано відгалуження Doctor -> Pediatrician. У програмі використано механізми Java: extends, super(...), @Override і final, а кожен клас має власний стан. JOL-дослідження підтвердило, що об'єкт CardiacSurgeon містить успадковані та власні поля в одному об'єкті JVM.